

目录

- 1 实验内容及要求
- 2 大模型给出的初始版本 v0
- 3 性能测试方法和设计原理
 - 3.1 测试工具选择
 - 3.2 测试用例设计
 - 3.2.1 测试数据生成
 - 3.2.2 测试项设计
 - 3.3 性能指标
 - 3.3.1 主要指标
 - 3.4 测试环境
 - 3.5 编译与运行
- 4 v0 版本的性能测试结果
- 5 针对 v0 版本的修改意见
- 6 大模型实现的 v1 版本
- 7 v1 版本性能测试结果
- 8 针对 v1 版本的修改意见
- 9 v2 版本的测试结果
- 10 性能测试结果分析
 - 10.1 三个版本的性能对比
 - 10.2 性能分析
- 11 作业总结
- 12 附录A：测试用例生成器
- 13 附录B：测试程序
- 14 附录C：Makefile 脚本

1 实验内容及要求

有一个字符串表示的地址，有三种可能：ipv4地址，ipv6地址，域名。写一个C语言的函数，来判断这个字符串是不是个域名。

```
1 bool isDomainName(const char* addr)
```

要求使用大模型编写，并且测试其性能，如果你认为性能不好，则提出修改方法让大模型进行修改，如果你认为大模型改的也不好，也可以自己再做修改。

注意前面的条件：已知这个字符串只有三种可能性，在此基础上判断是不是域名。

2 大模型给出的初始版本 v0

使用模型：GPT-5

Prompt:

```
1 有一个字符串表示的地址，有三种可能：ipv4地址，ipv6地址，域名。请你写一个C语言的函数 `bool
  isDomainName(const char* addr)`，来判断这个字符串是不是个域名。
2 注：已知这个字符串只有三种可能性，在此基础上判断是不是域名。
```

答复：见 [2023212539-张珂铭-performance.v0.cpp](#)。

```
1 #include "performance.h"
2 #include <cstring>
3 #include <cctype>
4
5 /**
6  * 判断字符串是否为域名
7  * 已知输入只有三种可能：IPv4地址、IPv6地址、域名
8  *
9  * @param addr 待判断的地址字符串
10  * @return true 如果是域名，false 如果是IP地址
11  */
12 bool isDomainName(const char* addr) {
13     if (addr == nullptr || *addr == '\0') {
14         return false;
15     }
16
17     int len = std::strlen(addr);
18
19     // IPv6地址包含冒号
20     for (int i = 0; i < len; i++) {
21         if (addr[i] == ':') {
22             return false; // 是IPv6地址
23         }
24     }
25
26     // 检查是否为IPv4地址
27     // IPv4地址只包含数字和点
28     bool hasLetter = false;
```

```

29
30     for (int i = 0; i < len; i++) {
31         if (std::isalpha(static_cast<unsigned char>(addr[i]))) {
32             hasLetter = true;
33             break;
34         }
35     }
36
37     // 如果没有字母, 可能是IPv4地址, 需要进一步验证
38     if (!hasLetter) {
39         // 简单检查: IPv4有3个点, 且都是数字
40         int dots = 0;
41         int digitCount = 0;
42         bool allValid = true;
43
44         for (int i = 0; i < len; i++) {
45             if (addr[i] == '.') {
46                 dots++;
47                 if (digitCount == 0 || digitCount > 3) {
48                     allValid = false;
49                     break;
50                 }
51                 digitCount = 0;
52             } else if (std::isdigit(static_cast<unsigned char>(addr[i]))) {
53                 digitCount++;
54             } else {
55                 allValid = false;
56                 break;
57             }
58         }
59
60         // 最后一段也要检查
61         if (digitCount == 0 || digitCount > 3) {
62             allValid = false;
63         }
64
65         // 如果有3个点且格式正确, 是IPv4
66         if (dots == 3 && allValid) {
67             return false; // 是IPv4地址
68         }
69     }
70
71     // 否则是域名
72     // 域名包含字母、数字、连字符和点
73     for (int i = 0; i < len; i++) {
74         char c = addr[i];
75         if (!std::isalnum(static_cast<unsigned char>(c)) && c != '.' && c != '-'
76             ' && c != '_' ) {
77             return false; // 包含非法字符
78         }
79     }

```

```
80 |         return true;    // 是域名
81 |     }
```

3 性能测试方法和设计原理

3.1 测试工具选择

我采用 **Google Benchmark** 作为性能测试框架，它提供纳秒级的高精度时间测量，并且会自动调整迭代次数，是 C++ 性能测试的工业标准，广泛应用于各类开源项目。

3.2 测试用例设计

3.2.1 测试数据生成

我编写了 Python 脚本自动生成 300 个测试用例，包括：

- **IPv4 地址**（100个）：使用 Python `ipaddress` 模块随机生成有效的 IPv4 地址
- **IPv6 地址**（100个）：随机生成 128 位整数并转换为 IPv6 格式
- **域名**（100个）：使用 Faker 库生成真实格式的随机域名

生成脚本核心代码：

```
1 | import random
2 | from faker import Faker
3 | import ipaddress
4 |
5 | # 生成 IPv4
6 | addr = str(ipaddress.IPv4Address(random.randint(0, 2**32 - 1)))
7 |
8 | # 生成 IPv6
9 | addr = str(ipaddress.IPv6Address(random.randint(0, 2**128 - 1)))
10 |
11 | # 生成域名
12 | addr = fake.hostname()
```

测试用例存储格式：`test_cases.txt`（Tab 分隔）

```
1 | address expected_type
2 | www.example.com DOMAIN
3 | 192.168.1.1 IP
4 | 2001:db8::1 IP
```

3.2.2 测试项设计

本实验设计了 5 个核心测试项：

测试项	目的	说明
BM_Domain	基准测试	测试标准域名的判断性能
BM_IPv4	基准测试	测试 IPv4 地址的判断性能
BM_IPv6	基准测试	测试 IPv6 地址的判断性能
BM_Mixed	混合场景	使用 300 个真实用例，模拟实际使用场景
BM_Correctness	正确性验证	验证所有测试用例的判断准确性

3.3 性能指标

3.3.1 主要指标

- 1. **Time（耗时）**：每次函数调用的平均时间（纳秒）
 - 越低越好，反映算法的时间复杂度
- 2. **CPU Time**：CPU 实际执行时间
 - 与 Time 相近表示 CPU 密集型操作
- 3. **Iterations（迭代次数）**：
 - 框架自动调整的执行次数
 - 次数越多说明单次执行越快
- 4. **items_per_second（吞吐量）**：
 - 每秒可处理的地址数量（百万次/秒）
 - 反映系统的处理能力

3.4 测试环境

- 操作系统：macOS (Darwin 24.6.0)
- 处理器：Apple Silicon (8核)
- 编译器：Clang++ (Apple)
- 优化级别：`-O2`
- C++ 标准：C++11
- Google Benchmark：v1.9.4

3.5 编译与运行

编译命令：其中 performance.cpp 需要根据实际程序名修改。

```
1 g++ -O2 -Wall -Wextra -std=c++11 -Isrc \  
2 -I/opt/homebrew/include \  
3 -o bench src/performance.cpp test/benchmark_test.cpp \  
4 -L/opt/homebrew/lib -lbenchmark -lpthread
```

运行测试：

```
1 ./bench
```

核心测试代码示例：

```
1 // 标准域名测试  
2 static void BM_Domain(benchmark::State& state) {  
3     const char* addr = "www.example.com";  
4     for (auto _ : state) {  
5         benchmark::DoNotOptimize(isDomainName(addr));  
6     }  
7 }  
8 BENCHMARK(BM_Domain);  
9  
10 // 混合场景测试  
11 static void BM_Mixed(benchmark::State& state) {
```

```
12     LoadTestCases(); // 加载300个测试用例
13     size_t index = 0;
14     for (auto _ : state) {
15         const auto& tc = g_test_cases[index % g_test_cases.size()];
16         benchmark::DoNotOptimize(isDomainName(tc.address.c_str()));
17         index++;
18     }
19     state.SetItemsProcessed(state.iterations());
20 }
21 BENCHMARK(BM_Mixed);
```

4 v0 版本的性能测试结果

2025-10-28T20:44:17+08:00
Running ./bench
Run on (8 X 24 MHz CPU s)
CPU Caches:
 L1 Data 64 KiB
 L1 Instruction 128 KiB
 L2 Unified 4096 KiB (x8)
Load Average: 2.32, 2.08, 2.23

Benchmark	Time	CPU	Iterations
BM_Domain	13.1 ns	13.1 ns	54151485
BM_IPv4	16.6 ns	16.6 ns	41740956
BM_IPv6	4.19 ns	4.19 ns	177464989
加载测试用例：300 个			
BM_Mixed	16.4 ns	16.4 ns	42994902 items_per_second=61.0372M/s
BM_Correctness/iterations:1	1959 ns	2000 ns	1

从测试结果可见，v0 版本由于需要遍历整个字符串检查冒号 → 再遍历检查字母 → 可能再遍历验证IPv4 → 最后遍历验证域名字符，最多 4 次遍历，导致执行时间较长。

5 针对 v0 版本的修改意见

大模型拿到题目给出的实现方法较为简单，当然一方面可能是因为我使用的提示词也比较简单。大模型采用的是排除法：首先排除IPv6地址，然后尝试排除IPv4地址，如果都不是IP地址，则认为是域名。

- 排除 IPv6: 通过检查字符串中是否包含冒号 (:) 来判断。
 - 策略: `if (addr[i] == ':') { return false; }`
- 排除 IPv4:
 - 初筛: 检查是否有字母。如果没有字母，则进入进一步的IPv4格式检查。
 - 细节: 检查点 (.) 的数量是否为3，并简单检查每个段是否只包含 1 到 3 位数字 (0-9)。
 - 策略: 通过点数和每段数字个数的简单逻辑判断。
- 判断为域名: 如果不是IPv6且不是IPv4，则认为是域名。最后还增加了对域名合法字符的简单检查（字母、数字、点、连字符、下划线）。

但是，由于“已知输入只有IPv4、IPv6、域名三者之一”，代码中域名字符检查是不必要的，并且对 IP 地址的检查也过于繁琐，对字符串进行了多次遍历（检查冒号、检查字母、检查数字和点、检查域名合法字符），虽然对于短字符串影响不大，但逻辑上可以更精简。对此，我向大模型再次提出：

可以直接基于假设条件，利用字母和冒号这两个字符进行判断，因为合法的 IPv4 地址不能有字母，合法的 IPv6 地址必然有冒号。按照以下核心特征进行判断：

- IPv6 地址：必然包含冒号 (:)。这是最高优先级、最可靠的 IP 地址特征。
- IPv4 地址：不包含冒号，且不包含字母。
- 域名：不包含冒号，且包含字母（或连字符、下划线等）。

6 大模型实现的 v1 版本

```
1 bool isDomainName(const char* addr) {
2     if (addr == nullptr || *addr == '\0') {
3         return false;
4     }
5
6     bool hasLetter = false;
7
8     // 遍历字符串，优先检查 IPv6 的最高优先级特征：冒号
9     for (const char* p = addr; *p != '\0'; ++p) {
10         char c = *p;
11
12         // 1. 如果包含冒号 ':', 则一定是 IPv6 地址 (IP地址)
13         // 必须优先判断冒号，以处理包含字母 a-f 的 IPv6 情况。
14         if (c == ':') {
15             return false; // 是 IPv6 地址
16         }
17
18         // 2. 记录是否存在字母 (用于区分 IPv4 和域名)
19         if (std::isalpha(static_cast<unsigned char>(c))) {
20             hasLetter = true;
21         }
22     }
23
24     // 执行到这里，字符串**不是 IPv6 地址** (不含冒号)。
25
26     // 3. 检查是否为域名
27     if (hasLetter) {
28         // 如果包含字母，且不是 IPv6，根据前提**它一定是域名**。
29         // (因为合法的 IPv4 不含字母)
30         return true;
31     }
32
33     // 4. 否则，它既不是 IPv6 (无冒号)，也不是域名 (无字母)。
34     // 根据前提，它只能是合法的 IPv4 地址 (只包含数字和点)。
35     return false; // 是 IPv4 地址
36 }
```

7 v1 版本性能测试结果

2025-10-28T20:45:20+08:00
Running ./bench
Run on (8 X 24 MHz CPU s)
CPU Caches:
 L1 Data 64 KiB
 L1 Instruction 128 KiB
 L2 Unified 4096 KiB (x8)
Load Average: 2.41, 2.13, 2.23

Benchmark	Time	CPU	Iterations
BM_Domain	7.16 ns	7.16 ns	101430165
BM_IPv4	5.27 ns	5.27 ns	133373981
BM_IPv6	5.66 ns	5.66 ns	131680430
加载测试用例：300 个			
BM_Mixed	9.34 ns	9.34 ns	75717423 items_per_second=107.042M/s
BM_Correctness/iterations:1	1708 ns	2000 ns	1

v1 版本采用单次遍历，消除了不必要的 IPv4 格式验证逻辑，在执行速度上相比 v0 版本显著提升。

8 针对 v1 版本的修改意见

大模型已经基本针对我的要求进行了修改。不过，它使用了一个循环来同时检查冒号和字母，这是可以利用标准库函数来简化的，因为标准库函数通常使用 SIMD 等技术在底层高度优化。

因此，我将 IPv6 检查改为使用 `std::strchr(addr, ':')` 代替手动循环。`strchr` 是专门设计用于此目的的，通常在底层比手写的循环更优化，并且明确了逻辑意图：检查是否存在冒号。然后将字母检查的逻辑作为第二步的独立循环。

在实现时我同时对理论效率进行了分析：

- 在最差情况下（例如一个不含冒号和字母的长 IPv4 地址），代码会执行两次遍历（一次 `strchr`，一次字母循环）。
- 在最佳情况下（例如一个 IPv6 地址），代码执行一次 `strchr` 立即返回。
- 对于大部分 IPv6 或 IPv4 地址，`strchr` 的效率很高。对于域名，`strchr` 快速返回 `nullptr`，然后字母循环可能会提前找到字母并返回。

我修改的代码如下：

```
1 bool isDomainName(const char* addr) {
2     if (addr == nullptr || *addr == '\0') {
3         return false;
4     }
5
6     if (std::strchr(addr, ':') != nullptr) {
7         return false; // 是 IPv6 地址
8     }
9
10    for (const char* p = addr; *p != '\0'; ++p) {
11        if (std::isalpha(static_cast<unsigned char>(*p))) {
12            return true; // 是域名
13        }
14    }
15    return false; // 是 IPv4 地址
16 }
```

考虑到使用多线程、并发等技术可能可以进一步提高程序性能，不过考虑到这个任务是针对一个单字符串的判断，且字符串长度通常较短（地址和域名长度有限制），多线程或并发处理在这里没有意义，反而会引入巨大的线程管理开销，可能导致性能远低于单线程的优化版本，所以，我最终采用单线程实现。

9 v2 版本的测试结果

```
2025-10-28T20:48:24+08:00
Running ./bench
Run on (8 X 24 MHz CPU s)
CPU Caches:
  L1 Data 64 KiB
  L1 Instruction 128 KiB
  L2 Unified 4096 KiB (x8)
Load Average: 1.79, 1.95, 2.13

=====
Benchmark                                Time                CPU    Iterations
-----
BM_Domain                                3.15 ns             3.15 ns    227581588
BM_IPv4                                  8.88 ns             8.88 ns    78305032
BM_IPv6                                  1.86 ns             1.86 ns    376722834
加载测试用例：300 个
BM_Mixed                                 6.41 ns             6.41 ns    106617927 items_per_second=155.983M/s
BM_Correctness/iterations:1              1499 ns            1000 ns         1
```

我实现的 v2 版本采用 `strchr` 优化的冒号检查 + 遇到第一个字母立即返回，执行时间上大大加快。

10 性能测试结果分析

10.1 三个版本的性能对比

测试项	v0 版本	v1 版本	v2 版本	v1 vs v0	v2 vs v1	v2 vs v0
BM_Domain	13.1 ns	7.16 ns	3.15 ns	↓ 45.3%	↓ 56.0%	↓ 76.0%
BM_IPv4	16.6 ns	5.27 ns	8.88 ns	↓ 68.3%	↑ 68.5%	↓ 46.5%
BM_IPv6	4.19 ns	5.66 ns	1.86 ns	↑ 35.1%	↓ 67.1%	↓ 55.6%
BM_Mixed	16.4 ns	9.34 ns	6.41 ns	↓ 43.0%	↓ 31.4%	↓ 60.9%
吞吐量	61.0 M/s	107.0 M/s	156.0 M/s	↑ 75.4%	↑ 45.8%	↑ 155.7%

10.2 性能分析

1. 混合场景性能

- 混合场景测试使用 300 个真实测试用例，最能反映实际应用中的性能表现：
- **v0 → v1**：从 16.4 ns 降至 9.34 ns，**性能提升 43.0%**
 - 主要原因：减少了遍历次数（从最多4次减少到1次）
 - 消除了不必要的 IPv4 格式验证逻辑
 - **v1 → v2**：从 9.34 ns 降至 6.41 ns，**性能提升 31.4%**
 - 主要原因：使用优化的标准库函数 `strchr`
 - `strchr` 通常使用 SIMD 指令优化，比手写循环快
 - **v0 → v2**：从 16.4 ns 降至 6.41 ns，**整体性能提升 60.9%**
 - 吞吐量从 61.0 M/s 提升至 156.0 M/s，**提升 155.7%**

2. 域名判断性能（BM_Domain）

域名判断在三个版本中都表现出持续优化：

版本	耗时	性能提升
v0	13.1 ns	基准
v1	7.16 ns	↓ 45.3%
v2	3.15 ns	↓ 76.0%

v2 版本表现最优的原因：

- `strchr` 快速确认不包含冒号（约 1-2 个 CPU 周期）
- 域名通常前几个字符就包含字母，可以快速返回
- 典型域名 `www.example.com` 在第一个字符 `w` 就可判断

3. IPv4 地址判断性能（BM_IPv4）

版本	耗时	分析
v0	16.6 ns	包含完整的 IPv4 格式验证，开销最大
v1	5.27 ns	最优！单次遍历，无字母即判定
v2	8.88 ns	<code>strchr</code> 开销 + 完整遍历无字母

在 IPv4 上，v1 的表现要优于 v2，主要是因为 IPv4 地址（如 `192.168.1.1`）不含冒号也不含字母，两者在处理时：

- v1 版本在单次遍历中同时检查冒号和字母，遍历完成后发现都不含，直接返回 false
- v2 版本虽然 `strchr` 快速确认无冒号，但还需要完整遍历一次检查字母
 - `strchr` 本身的函数调用开销（虽小但存在）
 - 对于 IPv4 这种“最后才能确定”的情况，两次操作反而不如 v1 的一次遍历

虽然 `strchr` 是优化的，但两次扫描的总开销大于 v1 的单次双条件扫描。

4. IPv6 地址判断性能（BM_IPv6）

IPv6 是三个版本中性能差异最明显的场景：

版本	耗时	分析
v0	4.19 ns	手写循环查找冒号，较快
v1	5.66 ns	单次遍历但检查两个条件
v2	1.86 ns	<code>strchr</code> 极致优化

v2 版本性能最优的原因：

- IPv6 地址（如 `2001:db8::1`）包含冒号，且冒号通常在前面位置
- `strchr` 使用 SIMD 指令可以一次比较多个字符
- 现代 CPU 的 `strchr` 实现可能使用：
 - SSE2/AVX2 指令一次比较 16/32 个字节
 - 硬件加速的字符串搜索指令

11 作业总结

本作业中，我与大模型进行交互，提出建议，不断迭代优化代码设计，在算法层面，通过减少遍历次数并利用题目前提简化逻辑，同时发挥标准库函数的威力，性能提升明显。同时利用 Google Benchmark 工具，让我对实际工程环境中的性能测试有了进一步的理解和熟悉。

12 附录A：测试用例生成器

```
1  #!/usr/bin/env python3
2
3  import random
4  from faker import Faker
5  import ipaddress
6
7  def generate_test_cases(num_per_type=100, output_file='test_cases.txt'):
8      fake = Faker()
9      test_cases = []
10
11     # 生成 IPv4 地址
12     for _ in range(num_per_type):
13         addr = str(ipaddress.IPv4Address(random.randint(0, 2**32 - 1)))
14         test_cases.append((addr, "IP"))
15
16     # 生成 IPv6 地址
17     for _ in range(num_per_type):
18         addr = str(ipaddress.IPv6Address(random.randint(0, 2**128 - 1)))
19         test_cases.append((addr, "IP"))
20
21     # 生成域名
22     for _ in range(num_per_type):
23         test_cases.append((fake.hostname(), "DOMAIN"))
24
25     # 随机打乱顺序
26     random.shuffle(test_cases)
27
28     with open(output_file, 'w', encoding='utf-8') as f:
29         f.write("address\texpected_type\n")
30         for address, expected_type in test_cases:
31             f.write(f"{address}\t{expected_type}\n")
32
33     print(f"成功生成 {len(test_cases)} 条测试用例到 '{output_file}'")
34
35 if __name__ == '__main__':
36     generate_test_cases()
```

13 附录B：测试程序

```
1  #include <benchmark/benchmark.h>
2  #include <fstream>
3  #include <sstream>
4  #include <vector>
5  #include <string>
6
7  #include "performance.h"
8
9  // 测试用例结构
10 struct TestCase {
```

```
11     std::string address;
12     std::string expected_type;
13 };
14
15 static std::vector<TestCase> g_test_cases;
16
17 static std::string trim(const std::string& str) {
18     size_t first = str.find_first_not_of(" \t\r\n");
19     if (first == std::string::npos) return "";
20     size_t last = str.find_last_not_of(" \t\r\n");
21     return str.substr(first, (last - first + 1));
22 }
23
24 // 加载测试用例
25 static void LoadTestCases() {
26     static bool loaded = false;
27     if (loaded) return;
28
29     std::ifstream file("test/test_cases.txt");
30     if (!file.is_open()) {
31         fprintf(stderr, "警告：无法打开测试用例文件 test/test_cases.txt\n");
32         loaded = true;
33         return;
34     }
35
36     std::string line;
37     // 跳过标题行
38     std::getline(file, line);
39
40     while (std::getline(file, line)) {
41         if (trim(line).empty()) continue;
42
43         std::stringstream ss(line);
44         std::string address, type;
45
46         if (std::getline(ss, address, '\t') && std::getline(ss, type)) {
47             address = trim(address);
48             type = trim(type);
49
50             if (!address.empty() && !type.empty()) {
51                 g_test_cases.push_back({address, type});
52             }
53         }
54     }
55
56     loaded = true;
57     fprintf(stderr, "加载测试用例：%zu 个\n", g_test_cases.size());
58 }
59
60 // 标准域名
61 static void BM_Domain(benchmark::State& state) {
62     const char* addr = "www.example.com";
```

```
63     for (auto _ : state) {
64         benchmark::DoNotOptimize(isDomainName(addr));
65     }
66 }
67 BENCHMARK(BM_Domain);
68
69 // IPv4地址
70 static void BM_IPv4(benchmark::State& state) {
71     const char* addr = "192.168.1.1";
72     for (auto _ : state) {
73         benchmark::DoNotOptimize(isDomainName(addr));
74     }
75 }
76 BENCHMARK(BM_IPv4);
77
78 // IPv6地址
79 static void BM_IPv6(benchmark::State& state) {
80     const char* addr = "2001:db8::1";
81     for (auto _ : state) {
82         benchmark::DoNotOptimize(isDomainName(addr));
83     }
84 }
85 BENCHMARK(BM_IPv6);
86
87 // 混合场景测试
88 static void BM_Mixed(benchmark::State& state) {
89     LoadTestCases();
90
91     if (g_test_cases.empty()) {
92         state.SkipWithError("没有可用的测试用例");
93         return;
94     }
95
96     size_t index = 0;
97     for (auto _ : state) {
98         const auto& tc = g_test_cases[index % g_test_cases.size()];
99         benchmark::DoNotOptimize(isDomainName(tc.address.c_str()));
100         index++;
101     }
102
103     state.SetItemsProcessed(state.iterations());
104 }
105 BENCHMARK(BM_Mixed);
106
107 // 正确性验证
108 static void BM_Correctness(benchmark::State& state) {
109     LoadTestCases();
110
111     if (g_test_cases.empty()) {
112         state.SkipWithError("没有可用的测试用例");
113         return;
114     }
```

```

115
116     int errors = 0;
117     for (auto _ : state) {
118         state.PauseTiming();
119
120         // 检查所有测试用例
121         for (const auto& tc : g_test_cases) {
122             bool result = isDomainName(tc.address.c_str());
123             bool expected = (tc.expected_type == "DOMAIN");
124             if (result != expected) {
125                 errors++;
126                 fprintf(stderr, "错误: %s - 期望: %s, 实际: %s\n",
127                     tc.address.c_str(),
128                     expected ? "DOMAIN" : "IP",
129                     result ? "DOMAIN" : "IP");
130             }
131         }
132
133         state.ResumeTiming();
134         benchmark::DoNotOptimize(errors);
135     }
136
137     if (errors > 0) {
138         state.SkipWithError("发现正确性错误!");
139     }
140 }
141 BENCHMARK(BM_Correctness)->Iterations(1);
142
143 BENCHMARK_MAIN();

```

14 附录C: Makefile 脚本

```

1 CXX = g++
2 CXXFLAGS = -O2 -Wall -Wextra -std=c++11
3 INCLUDES = -Isrc
4
5 .PHONY: all clean run help
6
7 all: bench
8
9 # 使用 pkg-config 获取 benchmark 的编译参数
10 BENCH_CFLAGS := $(shell pkg-config --cflags benchmark 2>/dev/null || echo "-I/opt/homebrew/include")
11 BENCH_LIBS := $(shell pkg-config --libs benchmark 2>/dev/null || echo "-L/opt/homebrew/lib -lbenchmark")
12
13 bench: src/performance_v3.cpp test/benchmark_test.cpp src/performance.h
14     $(CXX) $(CXXFLAGS) $(INCLUDES) $(BENCH_CFLAGS) -o bench \
15         src/performance_v3.cpp test/benchmark_test.cpp \
16         $(BENCH_LIBS) -lpthread
17     @echo "编译完成: bench"
18

```

```
19 # 运行 benchmark
20 run: bench
21     ./bench
22
23 # 生成测试用例
24 generate:
25     @echo "生成测试用例..."
26     cd test && python3 generate_cases.py
27     @echo "测试用例已生成到 test/test_cases.txt"
28
29 # 清理
30 clean:
31     rm -f bench *.o src/*.o test/*.o
32     @echo "清理完成"
```